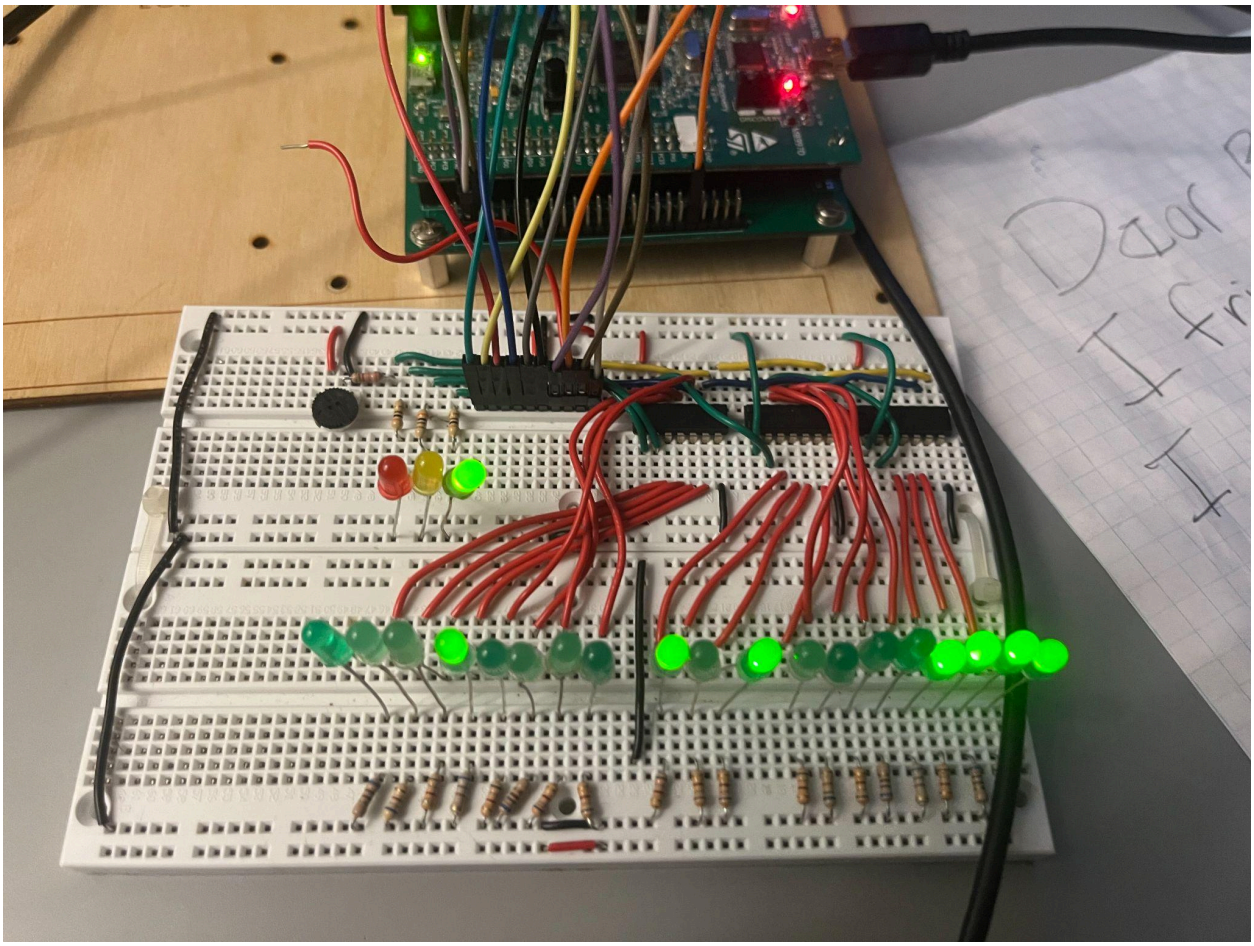
UNIVERSITY OF VICTORIA

Department of Electrical and Computer Engineering

ECE 455 - Real Time Computer Systems

Project 1: Traffic Light Systems



Date Submitted: March 6th, 2025

Arfaz Hossain, V00984826

Zachary Aldus, V00969072

Table of Contents

1 - Introduction **3**
1.1 Real-time operating systems (RTOS) 3
1.2 Objective 3
1.3 Design Constraints 3
1.4 Hardware Setup 4
1.4.1 Hardware Overview 4
1.4.2 Circuit Diagram 4
1.4.3 Circuit Diagram Description 5
2 - Initial Design Document **6**
2.1 Initial System Overview 6
2.2 Initial System Design 6
3 - Design Solution **7**
3.1 Initialization and Main 8
3.2 ADC Timer and Adjust_Traffic Task 8
3.3 Traffic Light Task 8
3.4 TL_Display Task 9
3.5 Car_Gen Task 10
3.6 Display_Street Task 11
4 - Software **12**
4.1 Initializations 12
4.1.2 prvSetupHardware() 12
4.1.2 ADC_setup () 13
poll_adc_function() 14
4.2 System Control 14
Adjust Flow Task 14
Traffic Light Task 15
Display Traffic Light Task 16
Car Generation Task 16
Display Street Task 17
4.3 Task and Queue Creation 18
Main Function 18
5 - Discussion **20**
5.1 Initial vs Final Design 20
5.2 Design Choices 20
5.2.1 Generation Algorithm 20
5.2.2 Computing Traffic Light Delay 21
5.2.3 Moving Car Algorithm 22
6 - Limitations and Improvements **23**
6.1 Limitations 23
6.2 Recommended Improvements 23
7 - Summary **24**
8 - References **25**
9 - Appendix **26**
9.1 Source Code 26

1 - Introduction

1.1 Real-time operating systems (RTOS)

Real-time operating systems (RTOS) are specialized software used in embedded systems computers designed for dedicated tasks - where multiple operations, or tasks, must be managed by a scheduler so that all are completed within strict time limits to avoid system issues. An RTOS ensures different tasks work together predictably to meet these timing constraints. This project shows how RTOS can manage multiple tasks and communicate with hardware elements to produce an effective embedded system.

1.2 Objective

The objective of this ECE455 Project 1 was to design and create a traffic light control system using the STM32F4 Discovery board and the FreeRTOS operating system. The system simulates traffic as a line of LEDs travelling at 1-3 LEDs per second down the line. After 8 LEDs, the cars will reach an intersection corresponding to 3 LEDs and their movement is determined by 3 LED traffic lights (red, yellow, and green). If the light is green, the cars will continue to move; if not, traffic will build up behind the intersection line (8th LED) until the light turns green. Once through the intersection, the cars will continue along 8 more LEDs with no movement constraints, as they have passed the intersection light. See Figure 1 below for setup of street LEDs.

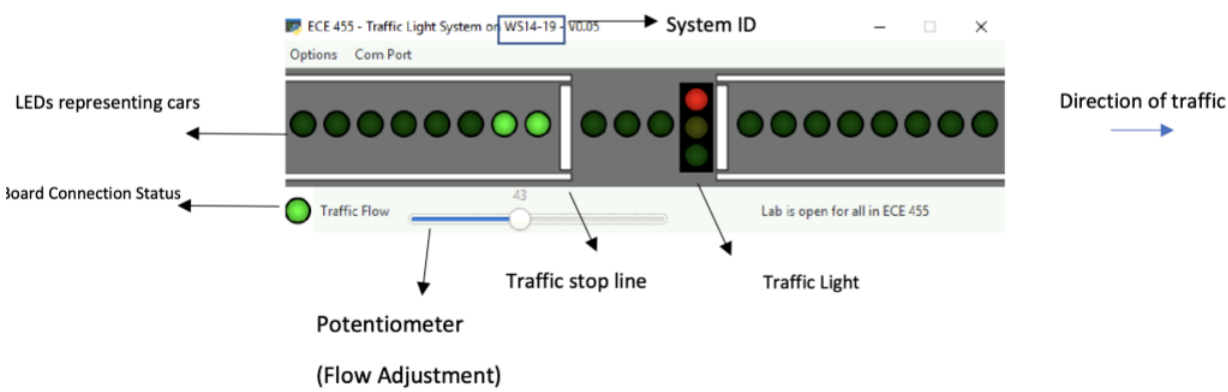


Figure 1.1 : User Interface for traffic light project [1]

1.3 Design Constraints

The rate of generated traffic is controlled using a potentiometer connected to the board's analog-to-digital converter (ADC), allowing our system to adjust the flow of traffic. A higher potentiometer setting produces a higher flow of traffic, and a lower setting produces a lower flow. To accommodate higher or lower traffic flow, the light delays of the red and green traffic LEDs will be proportional to the potentiometer value. When the traffic rate is maxed, the green light delay will be 2 times the red light delay and will depend on the minimum and maximum delay times, which can be changed in the variable definitions.

1.4 Hardware Setup

1.4.1 Hardware Overview

To design this system, the GPIO pins PC0-2 were used for the traffic light LEDs. GPIO pin PC03 was used to receive the potentiometer value, which was then converted by the board's ADC configuration. PC6-8 were used as input, clock, and reset, respectively, for a shift register daisy-chained with 2 additional shift registers. The 3 shift registers provided 24 outputs together, but only 19 would be used to drive 19 street LEDs (8 before intersection LEDs, 3 intersection LEDs, and 8 after intersection LEDs) that would display the generation and travel of the cars as the intersection lights shift and the generation is modified by the potentiometer.

1.4.2 Circuit Diagram

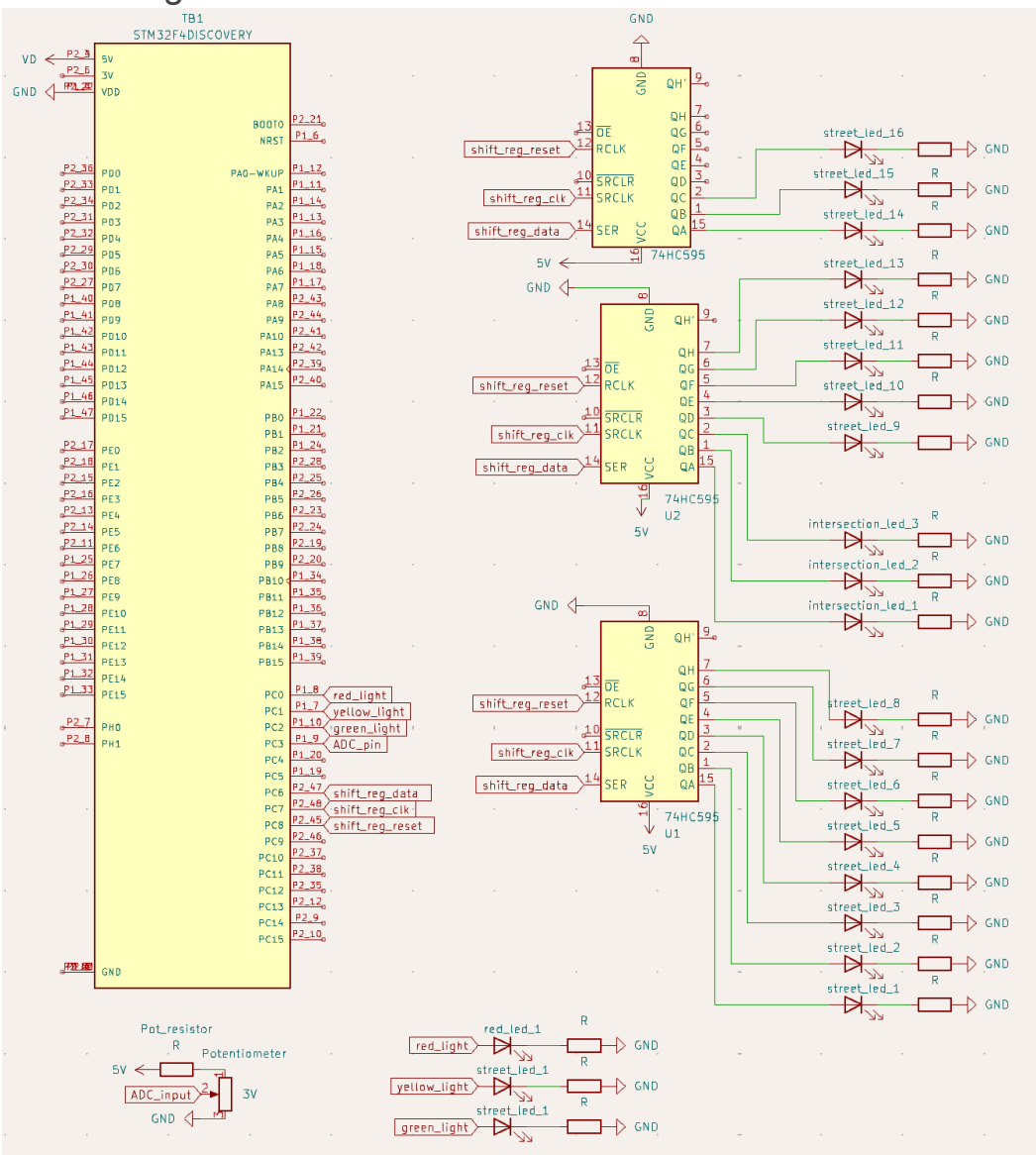


Figure 1.2: Circuit diagram of Project 1 traffic control

1.4.3 Circuit Diagram Description

The pin map to each component of the system is shown in the table below.

STM32F0 Port Pin	Signal	Notes
PC0	Red Light	Traffic Light
PC1	Amber Light	Traffic Light
PC2	Green Light	Traffic Light
		Traffic flow shift register
PC8	Shift Register Reset	Active Low, minimum 1us period
PC7	Shift Register Clock	Falling edge trigger
PC6	Shift Register Data	After clock falling edge data hold time of a minimum of 1 us
PC3	Potentiometer input	0 – 3 Volt Input

Figure 1.3 Port Configuration for Circuit Elements[1]

GPIO Ports PC0-2 are used to control the traffic lightLEDs red, yellow, and green, respectively. Each LED is connected to ground with a resistor to limit the current.

To output to the street LEDs and display cars moving, 3 daisy-chained 74HC595 shift registers are used. Using these registers allows the 19 LEDs to be controlled by 3 pins on the Discovery board: PC06-8, data, clock, and reset, respectively. The data pin sends a bit pattern to the shift registers. Each bit represents which LED should be turned on or off. With every clock pulse, the bits are shifted through the registers and output to the street LEDs. Each street LED also is connected to ground to limit the current

The potentiometer has 3 pins: power, ground and output. The power and ground pins are tied to the board's supply and ground, while the output pin is connected in series with a resistor to port PCO3. The potentiometer resistor limits the current, reducing the 5 V of the board to approximately 3 V. As the potentiometer is turned, the output voltage changes, producing different values ranging from 0-4095 because the ADC is configured to 12 bit resolution.. The digital ADC value determines the traffic flow rate, the speed at which cars are generated, and the duration of the green and red traffic light timing.

2 - Initial Design Document

2.1 Initial System Overview

The Traffic Control System demonstrates traffic flow as cars approach an intersection using FreeRTOS running on the STM32F4 Discovery board. The speed of the Traffic flow is controlled by a potentiometer connected to the ADC. The ADC rate determines how fast a car is generated and the duration of the red and green lights of the intersection. Cars are represented by LED outputs of 3 daisy chained shift registers, and traffic lights are represented by 3 other LEDs.

2.2 Initial System Design

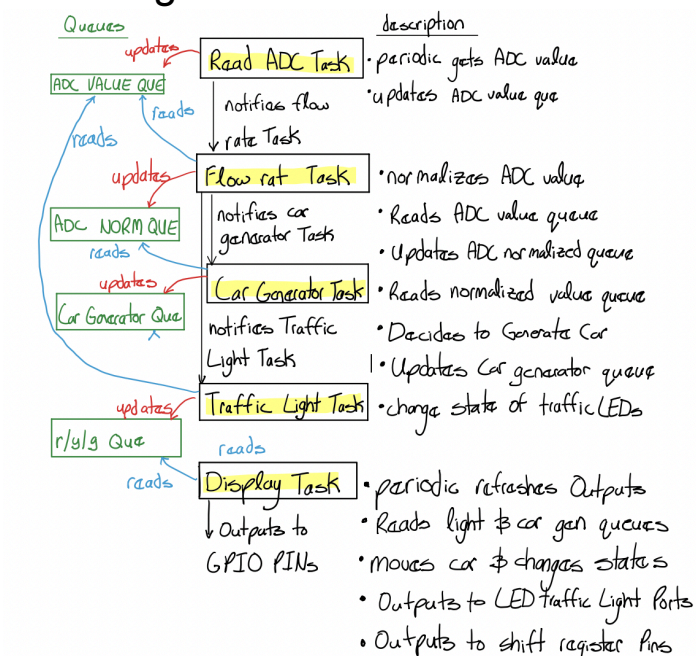


Figure 2.1 Initial Design Flow Chart

Read ADC Task
- Periodic reads potentiometer and produces ADC value
- Sends the ADC value to the ADC Value Queue.
- Notifies the Flow Rate Task when a new value is available.

Flow Rate Task
- Reads ADV Value Queue and normalizes the value
- Updates ADC normalized Queue

Car Gen Task
- Reads the normalized flow rate
- Decides if a Car should be generated
- Update car generator que

Traffic Light Task
- Changes State of the intersection traffic light

Display Task
- Outputs the state to the GPIO traffic light pins
- Moves Cars and Outputs to Shift Register data clock and reset

3 - Design Solution

This section explains the system design and communication between tasks and queues. The system overview Diagram can be seen below:

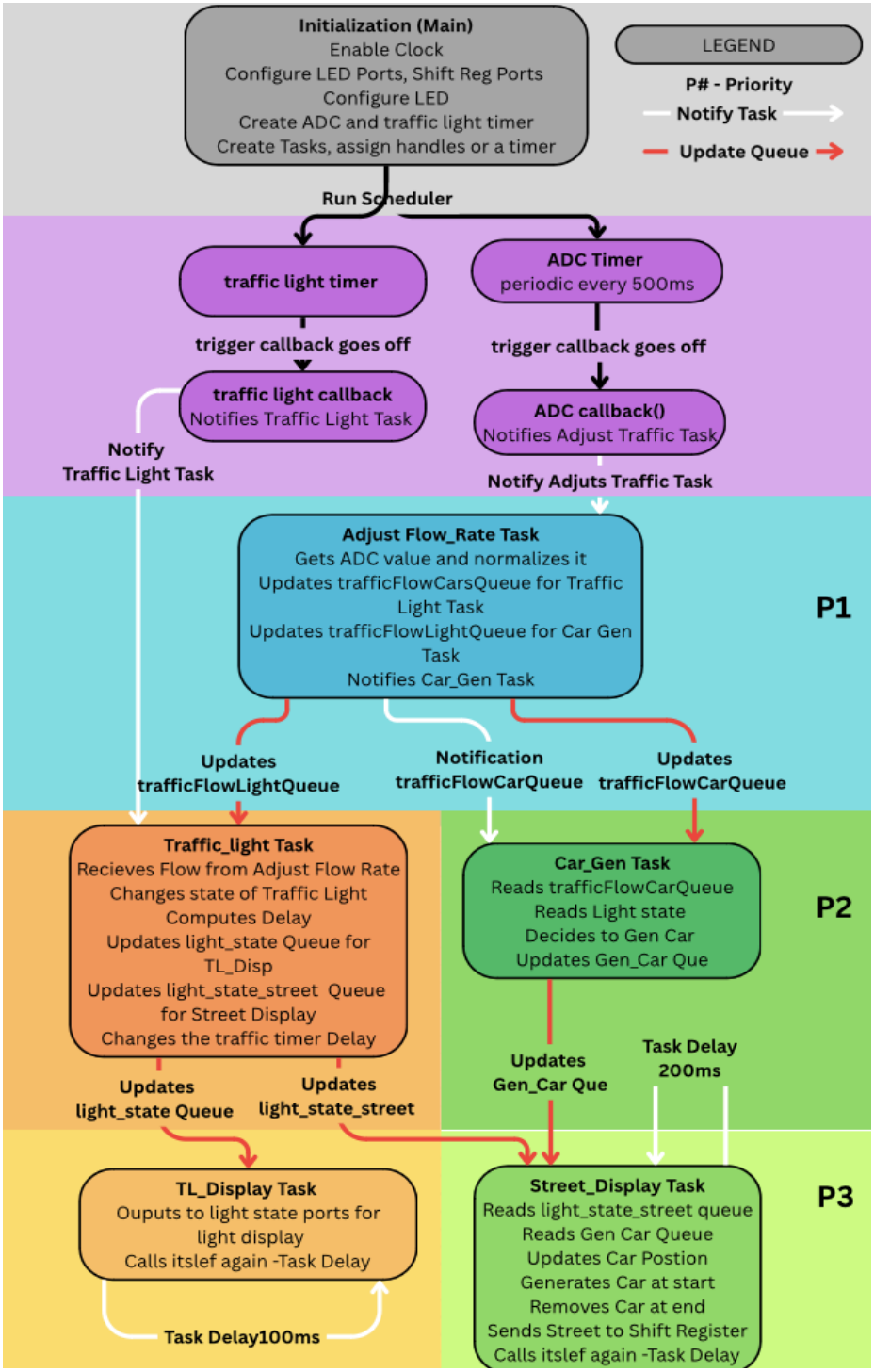


Figure 3.1 System Overview Diagram

3.1 Initialization and Main

In initialization, the AHB1 clock for GPIO Port C is enabled. GPIO pins for the traffic light LEDs and the shift register control are configured, and the ADC is configured with channel 3 and PC3 for the potentiometer. In main, queues, tasks, and timers are created and the scheduler is run.

3.2 ADC Timer and Adjust_Traffic Task

The ADC timer is periodic, triggered every 500 ms. When the ADC timer triggers, the ADC callback runs, waking the Adjust_Traffic task. The Adjust_Traffic performs an ADC conversion, reading the potentiometer and producing a digital value. The task then normalizes it by dividing by 4095, since the ADC is set to 12-bit resolution. The normalized value is between 0 and 1. And is written in 2 queues. The first queue is for the Traffic_Light task, which reads the flow rate (normalized ADC value) and determines the delay of the green and red traffic lights. A higher rate will produce a longer green than red. The other normalized ADC queue is for the Car_Gen task. This task converts the ADC-normalized value into a percentage of the probability of generating a car, then compares it to a randomly generated number. This ensures the rate at which cars are generated is random, not periodic. After updating the traffic flow queues, the Adjust_Traffic task notifies the car generation task that a new flow rate is added.

3.3 Traffic Light Task

The Traffic_Light task controls the traffic light states red, green, and yellow. It is triggered by the traffic light timer. The task will change its internal light variable to the next light state, and determine the delay for that light. The green light delay increases with traffic flow, and red light decreases. The yellow delay is constant. The light state is written to two queues. The first queue is used by the TL_Display task for updating the output pins of the traffic light LEDs. The second queue is used by the Display_Street task, which moves cars down the street and uses the light state to determine whether the light is red and the car needs to stop at the intersection.

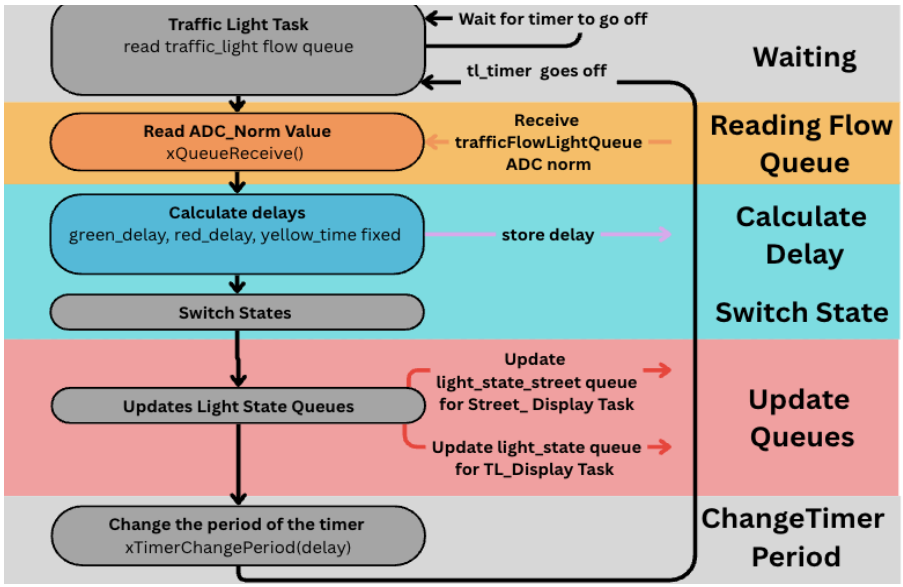


Figure 3.2: Traffic_Light Task Block Diagram

3.4 TL_Display Task

The TL_Display task just handles outputting to the traffic light LEDs. It reads the latest light state from its light state queue provided by the Traffic_Light Task. This task utilizes a helper function tl_change_state clears the GPIO outputs PC0-2, and then turns on the required lights output port. Clearing the pins before writing to the next state ensures that only one LED traffic light will be on at a time. The task then refreshes itself every 100ms. The TL_Display Block Diagram is shown below.

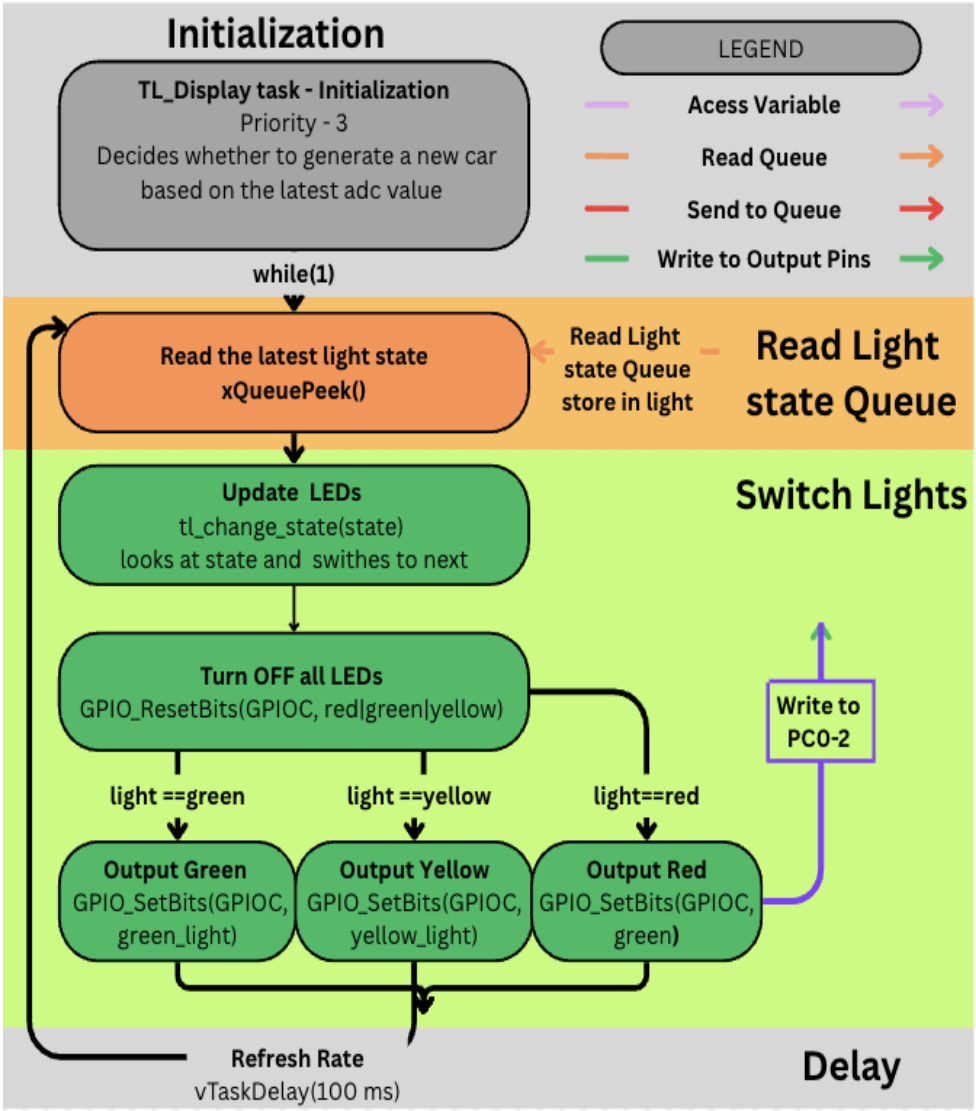


Figure 3.3: TL_Display Block Diagram

3.5 Car_Gen Task

The Car_Gen task uses the normalized ADC value read from the trafficFlowCarsQueue to decide if a car should be generated at the start of the street. The normalized ADC value is converted to a generation percentage and a random number is generated. These two values are compared and if the random number is less than the generated car percentage the task will update the gen_car_que . This algorithm is described in detail in the Discussion section of the report. This gen_car_que is used by the Display_Street Task to read and use when updating the street pattern and outputting to the three daisy chained shift registers. The Task runs periodically with a Task Delay of every 500ms allowing the rate of traffic to change effectively with a change in the potentiometer value. The Block Diagram for Car_Gen Task Can be seen below.

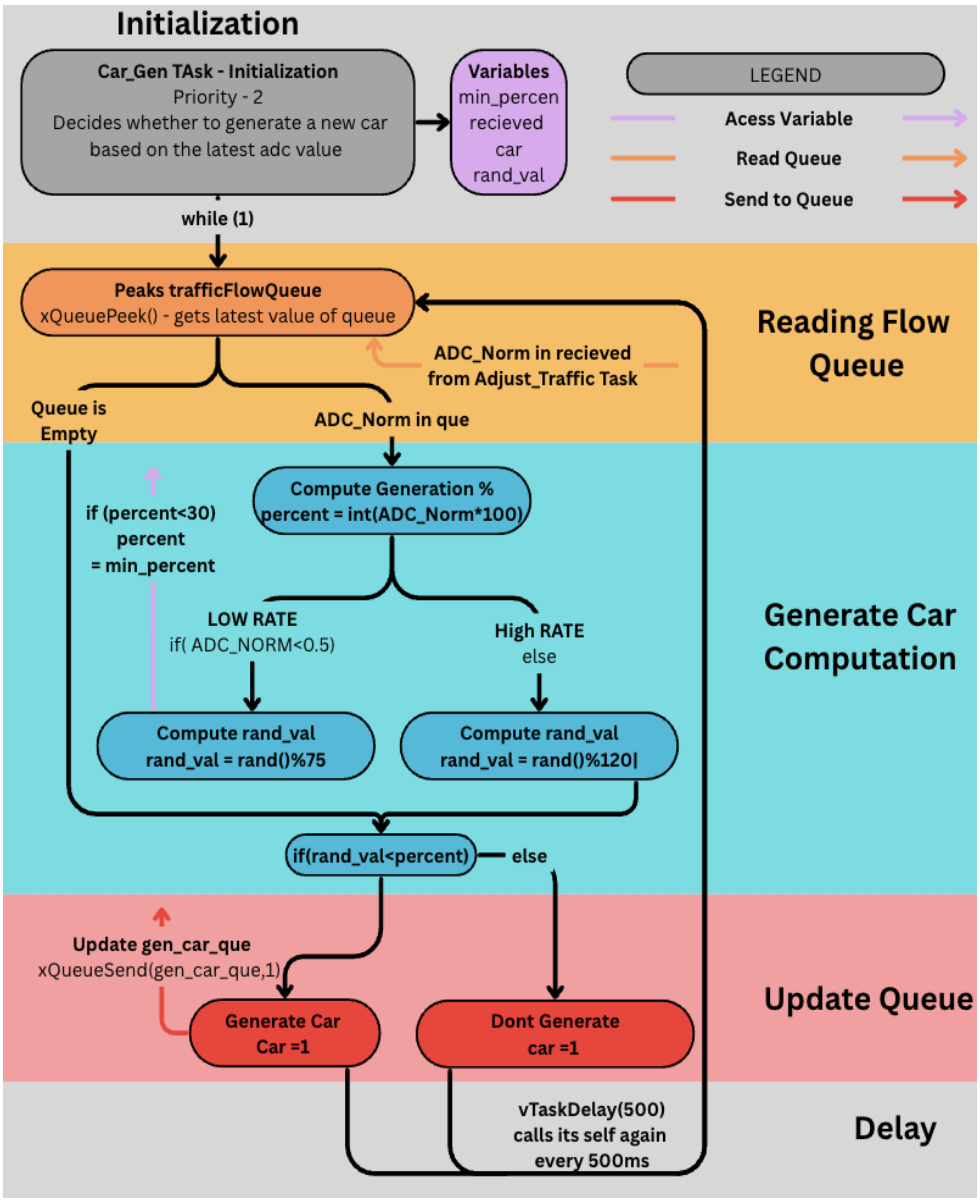


Figure 3.4: Car_Gen Task Block Diagram

3.6 Display_Street Task

The Display_Street task outputs to the street LEDs. It reads the latest traffic light state from its respective queue. It creates a 19-bit array for the 19 street LEDs. The task checks if the last LED has a car and clears it if the car has left. It loops backwards through the array. If the current spot is free and the spot behind has a car, it moves the car forward. The car is not shifted if it is at index 8 (start of the intersection) and the light is red. If the position in front is taken, the car does not move. After updating positions, the task sends the 19-bit road pattern to the daisy-chained shift registers via the GPIO data and clock. This updates the street's display LEDs. The task runs every 200 ms.

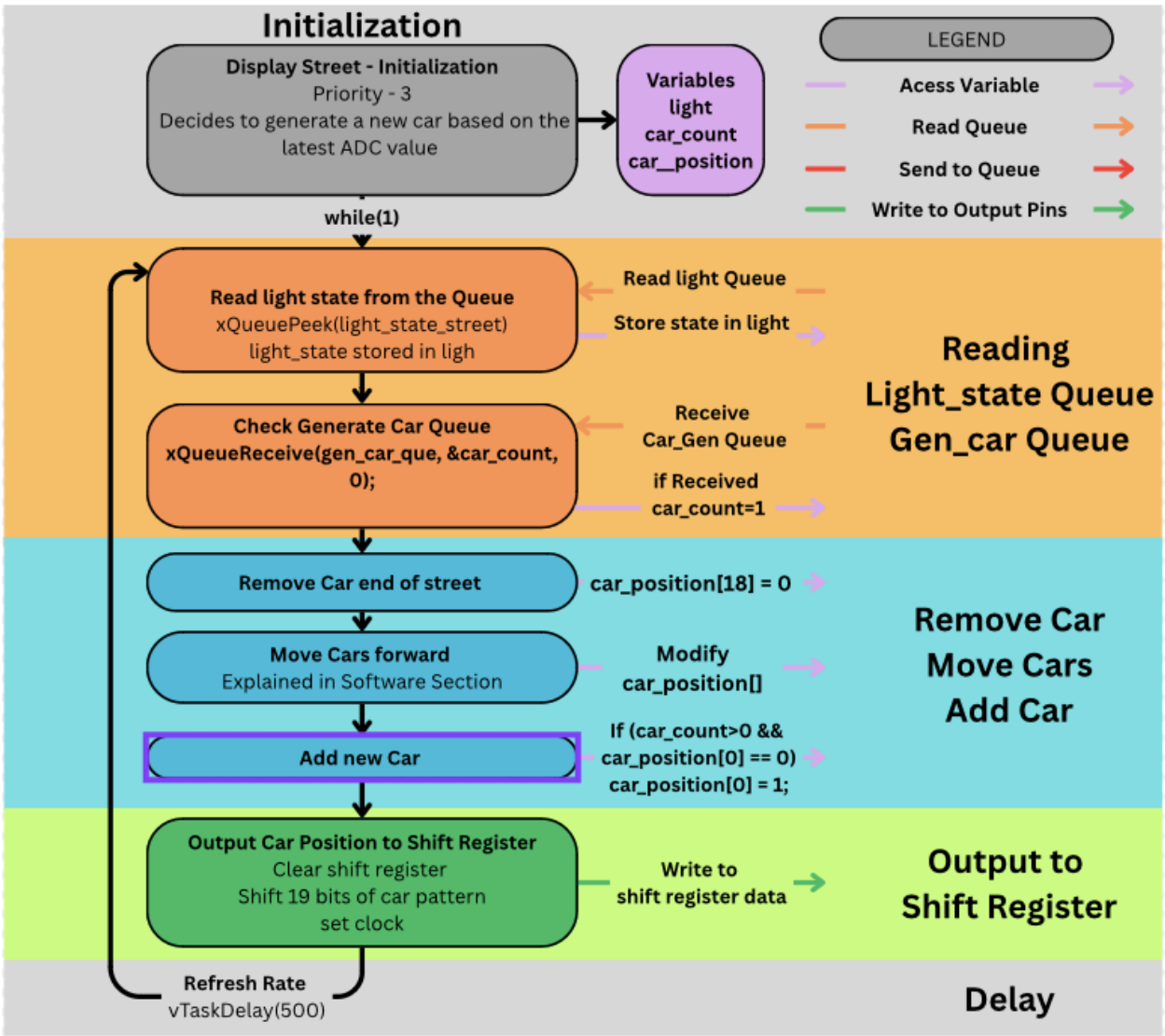


Figure 3.4: Display_Street Task Task Block Diagram

4 - Software

This section explains a detailed explanation of the software used to implement the design solution for Project 1.

4.1 Initializations

4.1.2 prvSetupHardware()

```
static void prvSetupHardware( void )
{
    /* Ensure all priority bits are assigned as preemption priority bits.
    http://www.freertos.org/RTOS-Cortex-M3-M4.html */
    NVIC_SetPriorityGrouping( 0 );

    // Enable GPIOC Clock
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
    GPIO_InitTypeDef GPIO_config;

    // Output pins
    GPIO_config.GPIO_Pin = red_light | green_light | yellow_light ;
    GPIO_config.GPIO_Mode = GPIO_Mode_OUT;      // Output
    GPIO_config.GPIO_OType = GPIO_OType_PP;     // Push Pull
    GPIO_config.GPIO_PuPd = GPIO_PuPd_DOWN;     // Pull Down
    GPIO_config.GPIO_Speed = GPIO_Speed_50MHz;  // Speed for shift register
    GPIO_Init(GPIOC, &GPIO_config);

    // Configure shift register output
    GPIO_InitTypeDef GPIO_config2;
    GPIO_config2.GPIO_Pin = shift_reg_clock | shift_reg_reset | shift_reg_data;
    GPIO_config2.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_config2.GPIO_OType = GPIO_OType_PP;
    GPIO_config2.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_config2.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOC, &GPIO_config2);
    ADC_setup();
}
```

Figure 4.1: prvSetupHardware()

The prvSetupHardware() function configures the hardware. The GPIO clock must be enabled before we configure the GPIO pins. For Port C, the clock is enabled on AHB1, allowing the pins we configure to use it. The red, yellow, and green lights PC01-02 are then enabled to drive the traffic light LEDs and configured as push-pull digital outputs with pull-down resistors. We then configure the shift register PC06-08. The data pin C06 of the shift register has a value of 0 or 1, indicating whether a car is in that position on the street. The clock pin C07 will shift the current

data bit on the rising edge of the clock, moving a car down the street. The reset C08 clears the output before writing a new sequence of bits.

4.1.2 ADC_setup ()

```
static void ADC_setup ( void ) {
    // ADC1 is on APB2
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
    // The GPIO pin (PC3) still needs AHB1
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
    GPIO_InitTypeDef GPIO_InitStruct;
    GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;      // Analog mode
    GPIO_InitStruct.GPIO_Pin  = GPIO_Pin_3;         // PC3
    GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;   // No pull resistor
    GPIO_Init(GPIOC, &GPIO_InitStruct);

    ADC_InitTypeDef ADC_InitStruct;
    ADC_InitStruct.ADC_ContinuousConvMode = DISABLE;
    ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b;
    ADC_InitStruct.ADC_NbrOfConversion = 1;
    ADC_InitStruct.ADC_ScanConvMode = DISABLE;
    ADC_InitStruct.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
    ADC_InitStruct.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
    ADC_Init(ADC1, &ADC_InitStruct);
    ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_84Cycles);
    ADC_Cmd(ADC1, ENABLE);  // Power on ADC1
}
```

Figure 4.2: ADC_setup()

ADC_setup() initializes the STM32F4's ADC1 so that the analog voltage of the potentiometer PC03 can be measured. We enable the clock for ADC1, which uses the APB2 bus. We initialize PC03 to analog mode, and no pull-up/down; otherwise, it would affect the voltage we are trying to read from the potentiometer. We disable continuous conversion and start conversion manually when we want to get an ADC value, saving clock time and running a conversion only when the system requires it. We set the resolution to 12-bit, meaning that the converted ADC value will be from 0-4095 and be a 12-bit output requiring a 16-bit integer to store this value. ExternalTrigConv is disabled because our software will start the conversion. We then select Channel 13 and set the sample time to 84 cycles, which controls how long the ADC waits for the voltage to build before performing the conversion. We experimented with higher cycle times, as the 84 cycles did not always get the full.

poll_adc_function()

```
static uint16_t poll_adc_function(void){
    uint16_t converted_data;
    // 1. Trigger the conversion
    ADC_SoftwareStartConv(ADC1);
    // 2. Wait for the End-of-Conversion flag
    while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
    // 3. Read the result (0-4095)
    converted_data = ADC_GetConversionValue(ADC1);
    return converted_data;
}
```

Figure 4.3: poll_adc_function()

poll_adc_function() is a helper function provided in the lab manual that performs an ADC conversion and returns the result. It starts the conversion with ADC_SoftwareStartConv() and waits for the conversion complete flag. Then reads the result with ADC_GetConversionValue() and returns the 12-bit integer (ADCconfigured to 12-bit resolution). The function is used in Adjust_Traffic when the ADC timer goes off, and the task wants to get the most recent potentiometer value. Despite the name, it isn't actually polling.

4.2 System Control

Adjust Flow Task

```
static void Adjust_Traffic(void *pvParameters) {
    uint16_t ADC_Result = 0;
    double ADC_Norm = 0;

    while(1) {
        ADC_Result = poll_adc_function();
        ADC_Norm = (double)ADC_Result / 4095.0;
        printf("ADC Raw: %u, Normalized: %.4f\n", ADC_Result, ADC_Norm);
        xQueueOverwrite(trafficFlowCarsQueue, &ADC_Norm);
        xQueueOverwrite(trafficFlowLightQueue, &ADC_Norm);
        xTaskNotifyGive(gen_handle);
    }
}
```

Figure 4.4: Adjust_Traffic ()

The Adjust_Traffic task is triggered by the ADC timer callback and gets the current ADC value using the poll_adc_function(). It will then normalize the ADC result and write it to two separate queues, one for the car generation task and one for the traffic light task. Because the queues are length 1 xQueueOverwrite() overwrites the last value. This way, neither the Car_Gen nor the Traffic_Light tasks ever use an old potentiometer reading. After updating the queues, the task

sends a notification to Car_Gen to indicate that a new value is ready. There's no delay in this task. The Adjust_Traffic wakes it so it relies on the ADC timer to be called again.

Traffic Light Task

```
static void Traffic_Light(void *pvParameters) {
    uint32_t delay = 0;
    double received = 0;
    uint8_t traffic_light = red;
    uint32_t green_delay = 0;
    uint32_t red_delay = 0;

    xQueueOverwrite(light_state, &traffic_light);
    xQueueOverwrite(light_state_street, &traffic_light);
    xTimerChangePeriod(tl_timer, pdMS_TO_TICKS(maximum_time), 0);

    while(1){
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        if (xQueueReceive(trafficFlowLightQueue, &received, 0));

            green_delay = (uint32_t)((minimum_time / 3.0) + received * ((2.0 *
maximum_time / 3.0) - (minimum_time / 3.0)));
            red_delay    = (uint32_t)((2.0 * minimum_time / 3.0) + received *
((maximum_time / 3.0) - (2.0 * minimum_time / 3.0)));

        switch (traffic_light)
        {
            case red:
                traffic_light = green;
                delay = green_delay;
                break;
            case green:
                traffic_light = yellow;
                delay = yellow_time;
                break;
            default:
                traffic_light = red;
                delay = red_delay;
        }
        xQueueOverwrite(light_state, &traffic_light);
        xQueueOverwrite(light_state_street, &traffic_light);
        xTimerChangePeriod(tl_timer, pdMS_TO_TICKS(delay), 0); }}
```

Figure 4.5: Traffic_Light ()

The Traffic_Light task runs the traffic light state changes. On startup, it sets the light to red, writes that to both light queues, and starts a one-shot timer with a maximum delay of 7500 ms. After that, it just waits; it blocks until that timer fires and sends it a notification. It is woken up by the traffic timer callback. When it wakes up, it reads the latest ADC normalized value from its traffic flow queue and recalculates the durations of the green and red phases. The formulas for the delays are explained in detail in the Discussion section of the report. The green delays are

proportional to the increase in the ADC value (more traffic) producing a longer green shorter red. Yellow always stays fixed at 1000 ms as per the lab manual. Once the delays are calculated, the light advances to the next state, writes the new state to both queues, and restarts the timer with the new period.

Display Traffic Light Task

```
static void TL_Display(void *pvParameters){
    uint8_t state = red;
    while(1){
        xQueuePeek(light_state, &state, portMAX_DELAY);
        tl_change_state(state);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Figure 4.6: TL_Display ()

TL_Display is solely responsible for driving the three LED pins based on the current state in the queue. It uses xQueuePeek() rather than xQueueReceive(), so it reads the value without removing it. Traffic_Light and Display_Street both need access to the same queue, and no coordination is needed between them. Once it has the state, the helper function tl_change_state() turns off all three LEDs simultaneously, then switches on the correct one. The task runs on a 100 ms polling loop, which means any light change is reflected in the hardware within a tenth of a second, fast enough that it's not noticeable.

Car Generation Task

```
static void Car_Gen(void *pvParameters){
    double received = 0;
    uint8_t car = 0;
    int percent = 0;
    int rand_val = 0;
    const int min_percent = 30;
    while(1) {
        if (xQueuePeek(trafficFlowCarsQueue, &recieved, 0) == pdTRUE) {
            percent = (int) (received * 100.0);

            if (recieved<0.5){
                rand_val = rand() % 75;
                if (percent < min_percent) percent = min_percent;
            } else { rand_val = rand() % 120; }

            if (rand_val < percent) {
                car = 1;
                printf("car generated");
                xQueueSend(gen_car_que, &car, 0);
            } else {
                car = 0;
```

```
            }
        }
        vTaskDelay(pdMS_TO_TICKS(500));
}
}
```

Figure 4.7: Car_Gen()

The Car_Gen task is notified by the Adjust Traffic Task and decides whether a car enters the street on each 500 ms cycle. It reads the current traffic flow value from the queue and converts it to a percentage from 0 to 100. It then picks a random number and checks whether that number falls below the percentage threshold. If it does, a car gets generated, and a message is placed in gen_car_que for the street display task to pick up. The car generation algorithm is described in detail in the report's discussion section.

Display Street Task

```
static void Display_Street(void *pvParameters){
    uint8_t car_position[19] = {0};
    uint8_t light = red;
    uint8_t car_count = 0;
    uint8_t cur_car_pos =0;

    while(1) {
        xQueuePeek(light_state_street, &light, 0);
        car_count = 0;
        if(xQueueReceive(gen_car_que, &car_count, 0) == pdTRUE) { }
        car_position[18] = 0;

        for(int i=18; i>0; i--){
            cur_car_pos = i-1;
            if(car_position[cur_car_pos] ==1 && car_position[i] ==0) {
                if(i==8 && light != green) { }
                else {
                    car_position[i] =1;
                    car_position[cur_car_pos] = 0;
                }
            }
        }

        if(car_count>0 && car_position[0] ==0) {
            car_position[0] = 1;
        }
        car_count = 0;
        GPIOC->ODR &= ~shift_reg_reset;
        for(volatile int i = 0; i < 1000; i++) { }
        GPIOC->ODR |=  shift_reg_reset;

        for(int i =18; i>=0; i--) {
            if(car_position[i] == 1) {
```

```
                GPIOC->ODR |= shift_reg_data;
            } else {
                GPIOC->ODR &= ~shift_reg_data;
            }
            GPIOC->ODR |=shift_reg_clock;
            for(volatile int i = 0; i < 1000; i++) { }
            GPIOC->ODR &= ~shift_reg_clock;
        }

        vTaskDelay(pdMS_TO_TICKS(200));
    }
}
```

Figure 4.7: Display_Street ()

Display_Street handles both the car movement logic and the physical LED output. It maintains a 19-element array, where each index corresponds to a position on the street: index 0 enters the street, and index 18 leaves it. Every 500 ms, it checks the current light state, checks whether a new car was generated, then steps through the array from back to front, moving cars forward. A car moves if the space ahead of it is empty, and if it's trying to cross the intersection from index 7 to 8, the light is green. If the light is red or yellow, cars back up behind position 7 until it turns green. After all positions are updated, a new car gets placed at index 0 if one was generated and that slot is free. The output stage resets all three shift registers by pulsing the reset line, then clocks out each bit of the array starting from index 18. The bits are in reverse order by the time the 19th clock pulse fires, since the first bit sent has been shifted all the way to the last output pin, keeping the LED order correct on the physical strip. Direct ODR register writes are used instead of the standard GPIO library calls because they're faster, which matters when clocking bits through a shift register quickly. This is also described in detail in the Discussion section of the report.

4.3 Task and Queue Creation

Main Function

```
int main ( void )
{
    prvSetupHardware();

    gen_car_que = xQueueCreate(1, sizeof(uint8_t));
    light_state = xQueueCreate(1, sizeof(uint8_t));
    light_state_street = xQueueCreate(1, sizeof(uint8_t));
    trafficFlowLightQueue = xQueueCreate(1, sizeof(double));
    trafficFlowCarsQueue = xQueueCreate(1, sizeof(double));

    xTaskCreate(
        Adjust_Traffic, "Adjust_Traffic", configMINIMAL_STACK_SIZE,
        NULL, 1, &adjust_traffic_handle
    );
```

```
    xTaskCreate(
        Traffic_Light,"Traffic_Light",configMINIMAL_STACK_SIZE,
        NULL, 2, &traffic_light_handle
    );
    xTaskCreate(
        TL_Display,"TL_Display",configMINIMAL_STACK_SIZE,
        NULL, 2, NULL);
    xTaskCreate(
        Car_Gen,"Car_Gen",configMINIMAL_STACK_SIZE,
        NULL, 3, &gen_handle
    );
    xTaskCreate(
        Display_Street,"Display_Street", configMINIMAL_STACK_SIZE,
        NULL, 3, NULL
    );

    tl_timer = xTimerCreate(
        "Traffic_Timer",
        pdMS_TO_TICKS(maximum_time),
        pdFALSE,
        NULL,
        tl_timer_callback
    );

    adc_timer = xTimerCreate(
        "Adc_timer",
        pdMS_TO_TICKS(500),
        pdTRUE,
        NULL,
        ADC_callBack
    );

    vTaskStartScheduler();
    return 0;
}
```

Figure 4.8: main (void)

The main (void) function brings up the system in three stages. First, prvSetupHardware() is called to configure the GPIO pins and ADC before anything else runs. Second, the five queues are created: one for generated cars, two for the light state (one for TL_Display and one for Display_Street), and two for the normalized ADC flow rate (one for Traffic_Light and one for Car_Gen). Splitting the light state and flow rate into separate queues for each consumer means no task ever has to wait on another to finish reading before it can get the value it needs. After the queues, the five tasks are created with their priorities assigned. Car_Gen and Display_Street are given priority 3, Traffic_Light and TL_Display are given priority 2, and Adjust_Traffic is given priority 1. Finally, two software timers are created: a one-shot timer for the traffic light (tl_timer) and a periodic timer for the ADC reads (adc_timer). The ADC timer is set to fire every 500 ms. Once everything is set up, vTaskStartScheduler() hands control to FreeRTOS and main never returns.

5 - Discussion

5.1 Initial vs Final Design

The final system design is similar to the initial design, but a few changes were made to improve communication between tasks. The First change was that we initially had two separate tasks: one to read and perform the ADC conversion, which would update an ADC value queue. The second task would then read this ADC value queue and write to its own queue, storing a normalized version of the ADC value. These 2 tasks were combined in the final design into the Adjust_Traffic task, which handled both performing an ADC conversion and normalizing the value, and removed the initial solely ADC-value queue.

The second change in the final design was removing the notification from the Adjust_Traffic task to the Traffic_Light task. In the initial design, we planned to have both the Adjust_Traffic, which handles the task's flow rate, and the traffic light timer wake the Traffic_Light task. This caused incorrect traffic signal timing, and the lights would change very quickly between green, yellow, and red. In the final design, the traffic light timer triggers only the Traffic_Light task, making sure the lights' delay is correct.

The final change from the initial design was to use separate queues for both the traffic flow value and the light state. This is because each queue would be used by 2 different Tasks. Having two copies of the same queue allowed the separate tasks to have their own resources and read the same data without interfering with each other. For example, for the traffic flow queues, both the Traffic_Light task and the car Car_Gen task used the ADC-normalized value.

5.2 Design Choices

5.2.1 Generation Algorithm

The Car_Gen task decides when a new car should be generated at the beginning of the street. The task reads the normalized ADC value from the trafficFlowCarsQueue and converts it into a generation percentage, producing a value between 0 and 100. percent = (int) (received * 100.0);

A random number is generated. This number is then compared to the calculated generation percentage. If the random number is less than the generation percentage, a signal is written to the gen_car_que for use by the Display_Street task when updating the street display.

We initially used rand_val = rand()%100 to generate the random value, but found that at high potentiometer readings near max, this comparison would produce a 100% generated car percentage that was not random. Additionally, at a low reading of the potentiometer, we found that the generated percentage would be 0%, and no car would be generated. To solve this, when the received ADC normalized value was 0.5 (50% spawn rate), we computed a random number with rand_val = rand()%120 to slightly reduce the 100% spawn rate, keeping it fast but

still random. To solve the issue at the minimum potentiometer value, we asked if percent was < min_percent, set percent = min_percent.

This provided a minimum spawn generation percent that allowed for cars to be generated at the minimum potentiometer reading. The minimum spawn generation was tested at different values before we settled on 30%. Additionally, to increase the minimum spawn generation, which was still very low, we used rand_val = rand()%75 to slightly boost generation speed while maintaining randomness.

5.2.2 Computing Traffic Light Delay

The green light delay increases as traffic flow increases, while the red light delay decreases. When traffic is at its peak, the green light should be two times the red light. At minimum flow the red and green light are swapped. We defined a minimum_delay = 3500ms and maximum_delay = 7000ms for the entire delay of the red, yellow, and green lights. These values were tweaked so we were happy with the output at both minimum and maximum flow rates

At the minimum car generation rate the green and red light will be

*red_delay = 2*green_delay*

green_delay + red_delay = minimum_time

Using the above equations we solve for green_delay and red delay

*green_delay + 2 * green_delay = minimum_time*

green_delay = minimum_time / 3

*red_delay = 2 * (minimum_time / 3)*

At maximum the green light equations are flipped

*green_delay = 2 * maximum_time / 3*

red_delay = maximum_time / 3

To incorporate the ADC flow rate into the green delays, we compute the difference between the max delay and the green_delay above. We multiply the difference by the flow rate and add to the original delay. At the minimum car generation rate the green light will be:

*green_delay = green_delay + flow rate * (difference btw max and green delay)*

*green_delay = (minimum_time / 3.0) + flow_rate * ((2 * maximum_time / 3.0) - (minimum_time / 3.0))*

At the minimum car generation rate the green light will be 2 times green light

*red_light =2*red_light +flow* difference*

*red_delay = (2 * minimum_time / 3.0) + flow * ((maximum_time / 3.0) - (2 * minimum_time / 3.0))*

For the max car generation rate, the equations for red and green light are flipped.

5.2.3 Moving Car Algorithm

The Display_Street task moves the cars along the LEDs that denote the street. The task uses a 19-element array, car_position [19], to store, at each index, whether the corresponding light should be on or off, indicating whether a car is there. The task is periodic, using task delay to call itself back every 500 ms, at the speed of the cars. The Task reads the light_state_street queue to know if the light is currently red, yellow, or green.

We first clear the last spot of the array, a car is leaving the system, car_position [19] = 0;

The array is updated by looping backwards from index 19 to 0 over the car positions. For each index i, the system checks whether there is a car at index i and at position i-1. If a car is in position i-1 behind and the spot in front is at index i, the car is allowed to move to the next spot, so we clear i-1 and write 1 to i in the car_position array.

The constraint on their movement is that if we are at index 8, then the car behind at index 7 is on the intersection line. We only allow it to move if the light state is green; otherwise, it will have to wait.

The final step for the array is to check the car_gen_que produced by the Car_Gen Task to see if a new car should be generated. If there is a car to be generated, we will write a 1 to the first index of the car_position array, car_position[1] =0

The Task will now write the array to the shift registers, first clearing them using the reset pin, which is active low. We then send each bit of the array from highest to lowest into the shift register by setting the data pin to high if the car_position array has a 1 and low if it has a 0. We then fire the clock of the shift registers between each bit of the array

6 - Limitations and Improvements

6.1 Limitations

The current system has some limitations which affect how closely it reflects real traffic. The car movement speed is fixed regardless of the potentiometer reading. All cars move at the same rate, which does not reflect real traffic, such as slowdowns due to congestion.

Additionally the street is a single-lane. There is no concept of opposing traffic from another lane using the intersection, which could add cars turning right and might cause a preemptive stop even if the light was green. Because the queue length is one for the car_gen_que the system to system does not know how many cars have backed up behind the intersection, so it cannot make the green light longer to respond to the backup of traffic as a traffic light control system would in the real world.

These limitations mean the system does not accurately show real world traffic conditions or allow for the system to have a more adaptive response. Adding additional variables to the project scope would increase the difficulty of the project but would make it cooler.

6.2 Recommended Improvements

Even though it would be cool to see a second intersecting street, it would require its own set of traffic lights and street LEDs running perpendicular to the first. We recommend adding a single LED perpendicular to the intersection to indicate a car turning right. An additional street would require another set of shift registers and another set of LED lights, which would drastically increase the amount of circuit board for the project. The single LED denoting a car turning right on the street could easily reuse the car generation algorithm and only require one more pin on the board, and the code could be added more simply to the display street function to denote a car turning right into the intersection.

Car speed could be made dynamic and tied to the potentiometer value, rather than keeping it fixed at 500 ms. At the minimum car generation rate, cars would likely drive faster and move more slowly; at maximum, they would likely drive slower. This implementation would require a similar delay calculation to the traffic light control, which would need to use a timer rather than a simpler task delay.

A system that dynamically adjusts light timing based on the buildup of cars at the intersection, rather than the potentiometer value, would better mimic a real-world traffic light system. This could be implemented by adding a build-up length queue, likely produced by the display street task for the traffic light task to use the amount of build-up behind the intersection into its delay calculation.

7 - Summary

Project 1: Traffic Light System implemented a traffic light control system on the STM32F4 Discovery board using FreeRTOS. The system simulates vehicles travelling down a 19-LED street, stopping at an intersection controlled by red, yellow, and green traffic light LEDs. Traffic flow rate and light timing are both driven by a potentiometer read through the onboard ADC.

The 5 tasks implemented in the final design

- Adjust_Traffic reads and normalizes the potentiometer value and distributes it to the rest of the system.
- Traffic_Light runs switches between light states and computes the red and green delays proportional to the ADC value;
- TL_Display outputs to the traffic light LEDs the current light state
- Car_Gen decide whether a car enters the street each cycle; and
- Display_Street uses a 19-element car array to hold cars' positions on the street and uses the gen_car_que to add a car or the light_state que to stop the car movement at the intersection.

The final design combines ADC read and normalization into a single task, unlike the initial document. It also creates 2 queues for both the ADC flow rate and the light state to avoid conflicts with tasks that need to use both queues. It finally breaks the display into 2 tasks rather than the initial design's task that handled both the traffic light and the street display.

The project was a good introduction to RTOS scheduling and provided a deeper understanding of task prioritization, communication through queues, software timers, and task notifications, and how these elements can be combined to produce a responsive, hardware-driven real-time system.

8 - References

[1] Department of Electrical and Computer Engineering, University of Victoria, "ECE 455 Lab Manual," ECE 455 Real Time Computer Systems, University of Victoria. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/ECE455-Lab-Manual.pdf

[2] Department of Electrical and Computer Engineering, University of Victoria, "ECE 455 Orientation Slides," ECE 455 Real Time Computer Systems, University of Victoria. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/ECE%20455%20Orientation.pdf

[3] Department of Electrical and Computer Engineering, University of Victoria, "ECE 455 Parts Kit," ECE 455 Real Time Computer Systems, University of Victoria. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/ECE455%20Parts%20kit.pdf

[4] Lite-On Technology Corporation, "LTC-4727JR Datasheet," University of Victoria ECE 455. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/ltc-4727jr.pdf

[5] NXP Semiconductors, "74HC164 8-bit Serial-in/Parallel-out Shift Register Datasheet," University of Victoria ECE 455. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/74hc164.pdf

[6] Department of Electrical and Computer Engineering, University of Victoria, "ECE 455 Lab 1 Code Template," ECE 455 Real Time Computer Systems, University of Victoria. [Online]. Available: https://www.ece.uvic.ca/~brent/ece455/Lab1_Code.zip

[7] Real Time Engineers Ltd., "FreeRTOS - Market leading RTOS," FreeRTOS.org. [Online]. Available: https://www.freertos.org

9 - Appendix

9.1 Source Code

For a more detailed explanation of code please see the **4 - Software** section of this report

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include "stm32f4_discovery.h"
#include <stdlib.h>
/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

// Traffic Light States
#define yellow 1
#define green 2
#define red 0
#define car_speed 500

#define adc_pin GPIO_Pin_3
#define red_light GPIO_Pin_0
#define yellow_light GPIO_Pin_1
#define green_light GPIO_Pin_2
#define shift_reg_data GPIO_Pin_6
#define shift_reg_clock GPIO_Pin_7
#define shift_reg_reset  GPIO_Pin_8
#define mainQUEUE_LENGTH 5

// default yellow time (const)
// #define mainQUEUE_LENGTH 100
#define yellow_time 1000
#define minimum_time 3500
#define maximum_time 7000
#define default_light_time 2000
// #define sample_rate_pot 500

//////////////////////////////////////////////////////////
//queues
static xQueueHandle gen_car_que; // uint8_t 1 gen car 0 no car
//light state handles hold current light state
//uint8_t holds state of the light
static xQueueHandle light_state_street;
```

```
static xQueueHandle light_state;
//flow queues hold the normalized ADC rate
static xQueueHandle trafficFlowLightQueue;
static xQueueHandle trafficFlowCarsQueue;

// Task Handlers
static TaskHandle_t adjust_traffic_handle;
static TaskHandle_t traffic_light_handle;
static TaskHandle_t gen_handle;

// Timers
static TimerHandle_t tl_timer;
static TimerHandle_t adc_timer;

// Functions Declarations
// static void Traffic_Light(void *pvParameters);
// static void ADC_callBack(TimerHandle_t xTimer);
// static void Adjust_Traffic(void *pvParameters);
static void Adjust_Traffic(void *pvParameters);
static void Traffic_Light(void *pvParameters);
static void TL_Display(void *pvParameters);
static void Car_Gen(void *pvParameters);
static void Display_Street(void *pvParameters);
static void ADC_callBack(TimerHandle_t xTimer);
static void tl_timer_callback(TimerHandle_t xTimer);

//////////////////////////////////////////////////////////
//hardware functions
static void prvSetupHardware( void );//sets up clocks and gpio pins
static void ADC_setup(void);
//adc poll helper function
//polls the adc reading from pot to get new rate of gen
static uint16_t poll_adc_function(void);
//changes the output of the TL LEDs
static void tl_change_state(uint8_t light);
static void shift_reg_clear(void) { }

/*-----------------------------------------------------------*/

int main(void)
{
   // Initialize Hardware
   prvSetupHardware();

   // Build Queues
   gen_car_que = xQueueCreate(1, sizeof(uint8_t));
   light_state = xQueueCreate(1, sizeof(uint8_t));                // 2
Queues for Light State
   light_state_street = xQueueCreate(1, sizeof(uint8_t));
```

```
   trafficFlowLightQueue = xQueueCreate(1, sizeof(double));            // 2
Queues for Traffic Flow
   trafficFlowCarsQueue = xQueueCreate(1, sizeof(double));

   // Build Tasks
   xTaskCreate(Adjust_Traffic,"Adjust_Traffic",configMINIMAL_STACK_SIZE,
NULL, 1, &adjust_traffic_handle);
   xTaskCreate(Traffic_Light,"Traffic_Light",configMINIMAL_STACK_SIZE, NULL,
2, &traffic_light_handle);
   xTaskCreate(TL_Display,"TL_Display",configMINIMAL_STACK_SIZE, NULL, 2,
NULL);
   xTaskCreate(Car_Gen,"Car_Gen",configMINIMAL_STACK_SIZE, NULL, 3,
&gen_handle);
   xTaskCreate(Display_Street,"Display_Street", configMINIMAL_STACK_SIZE,
NULL, 3, NULL);

   //one-shot timer for traffic light
   tl_timer =
xTimerCreate("Traffic_Timer",pdMS_TO_TICKS(maximum_time),pdFALSE,NULL,tl_tim
er_callback);
   //periodic timer for ADC
   adc_timer = xTimerCreate(
       "Adc_timer",
       pdMS_TO_TICKS(500),
       pdTRUE,
       NULL,
       ADC_callBack
   );
   //start ADC conversion

   vTaskStartScheduler();
   return 0;
}

//complete
//ADC_callBack - wakes the traffic adjust task when the adc timer goes off
//periodic timer every 500ms
static void ADC_callBack(TimerHandle_t xTimer) {
   xTaskNotifyGive(adjust_traffic_handle);
}

//complete
//tl_timer_callback calls itself back
//one shot timer
static void tl_timer_callback(TimerHandle_t xTimer) {
   xTaskNotifyGive(traffic_light_handle);
}

static void Adjust_Traffic(void *pvParameters) {
   uint16_t ADC_Result = 0;
```

```
   double ADC_Norm = 0;
   // check if notified then send, check
   while(1){
       //wait for notification

       //get ADC value
       ADC_Result = poll_adc_function();
       //normalize adc value
       ADC_Norm = (double)ADC_Result / 4095.0;
       printf("ADC Raw: %u, Normalized: %.4f\n", ADC_Result, ADC_Norm);
       //give to ques
       xQueueOverwrite(trafficFlowCarsQueue, &ADC_Norm);
       xQueueOverwrite(trafficFlowLightQueue, &ADC_Norm);
       // after send, notify traffic light and create traffic tasks
           //notify traffic_light task
           //dont think we need the traffic light task will deal with timer
       //xTaskNotifyGive(traffic_light_handle);
           //notify
       xTaskNotifyGive(gen_handle);
   }
}

//notified by the adjust traffic
static void Traffic_Light(void *pvParameters) {
   //initialization - runs once
   uint32_t delay = 0;
   double received = 0;
   uint8_t traffic_light = red; //start with red light
   uint32_t green_delay = 0;
   uint32_t red_delay = 0;

   xQueueOverwrite(light_state, &traffic_light);
   xQueueOverwrite(light_state_street, &traffic_light);
   xTimerChangePeriod(tl_timer, pdMS_TO_TICKS(maximum_time), 0);

   while(1){
       // once notified, do the calculation
           //might not need the pdMs could be set to zero
       ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

       //get the latest adc value
       if (xQueueReceive(trafficFlowLightQueue, &received, 0));

       // Calculations for light delay
       green_delay = (uint32_t)((minimum_time / 3.0) + received * ((2.0 *
maximum_time / 3.0) - (minimum_time / 3.0)));
       red_delay   = (uint32_t)((2.0 * minimum_time / 3.0) + received *
((maximum_time / 3.0) - (2.0 * minimum_time / 3.0)));
```

```
        //change light state
        switch (traffic_light)
        {
            case red:
                traffic_light = green;
                delay = green_delay;
                break;
            case green:
                traffic_light = yellow;
                delay = yellow_time;
                break;
            default:
                traffic_light = red;
                delay = red_delay;
        }
        // send light state to queues
        xQueueOverwrite(light_state, &traffic_light);
        xQueueOverwrite(light_state_street, &traffic_light);
        //delay task to call itself again
        xTimerChangePeriod(tl_timer, pdMS_TO_TICKS(delay), 0);
    }
}

//traffic light output task
    //should just run tl_change helper function based on the light state in
the que
static void TL_Display(void *pvParameters){
    uint8_t state = red;
    while(1){
        xQueuePeek(light_state, &state, portMAX_DELAY);
        tl_change_state(state);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

//traffic light changer helper fun
static void tl_change_state(uint8_t light){
    //reset all lights
    GPIO_ResetBits(GPIOC, red_light | green_light | yellow_light);
    //set light
    if(light == green){
        GPIO_SetBits(GPIOC, green_light);
    }else if(light == yellow){
        GPIO_SetBits(GPIOC, yellow_light);
    }else{
        GPIO_SetBits(GPIOC, red_light);
    }
}
```

```
//car_generator task
   //notified by the flow adjust task that a new adc value is ready
   //generates an array to drive the shift register
   //updates the generate car que
   //notifies the street display task
static void Car_Gen(void *pvParameters){
   double received = 0;
   uint8_t car = 0;
   int percent = 0;
   int rand_val = 0;
   const int min_percent = 30;
   //convert adc to a int that denotes the prob of car spawning ie 0.5 ->
50%

   while(1){
       if (xQueuePeek(trafficFlowCarsQueue, &recieved, 0) == pdTRUE) {
           percent = (int)(received * 100.0);
           if(recieved<0.5){
               rand_val = rand() % 75;
               if (percent < min_percent) percent = min_percent;
           }else{
               rand_val = rand() % 120;

           }



           if (rand_val < percent) {
               car =1; // gen car
               // only sends a message when the car is generated
               printf("car generated");
               xQueueSend(gen_car_que, &car, 0);
           } else {
               car = 0; // dont gen car

           }
       }
       ///run itself again every 500 ms
       vTaskDelay(pdMS_TO_TICKS(500));
   }
}

//street display task
static void Display_Street(void *pvParameters){
   uint8_t car_position[19] = {0};
   uint8_t light = red;
   uint8_t car_count = 0;
   uint8_t cur_car_pos =0;

   while(1){
```

```
        //get latest light state
        xQueuePeek(light_state_street, &light, 0);

        //want to count the number of cars received
        car_count =0;
        if(xQueueReceive(gen_car_que, &car_count, 0) == pdTRUE) {

        }

        car_position[18] = 0;
        //move car along by one led
        //i is the index of the street
        for(int i = 18; i>0;i--){
            cur_car_pos = i-1;
            //if the spot in front of the car is clear, move the car
            if(car_position[cur_car_pos] ==1 && car_position[i] ==0) {
                //want to check the its not blocked by the light
                //we want to stop the car between the 8th and 9th led index 7
and 8
                if(i==8 && light != green) {
                    //block car
                }else{
                    //we move the car
                    car_position[i] =1;
                    car_position[cur_car_pos] =0;
                }
            }
        }
    //add the generated car
    if(car_count>0 && car_position[0] ==0){
        //car to be generated and first led is clear
         car_position[0] = 1;
    }
    car_count = 0;

    //output array to shift register
    GPIOC->ODR &= ~shift_reg_reset; // clear shift reg
    //delay
    for(volatile int i = 0; i < 1000; i++){
    }
    GPIOC->ODR |=  shift_reg_reset;//sets shift reg high

    //send out bits
    for(int i =18; i>=0; i--){
        if(car_position[i] == 1){
            GPIOC->ODR |= shift_reg_data; //set the data pin high
        } else{
            GPIOC->ODR &= ~shift_reg_data; //set the data pin to low
        }
        // trigger clock
```

```
        GPIOC->ODR |=shift_reg_clock;//clock high
        // small delay
        for(volatile int i = 0; i < 1000; i++) { }
        GPIOC->ODR &= ~shift_reg_clock; //clcok low

    }

    //run itself again, refresh rate
    vTaskDelay(pdMS_TO_TICKS(200));
    }
}

void vApplicationMallocFailedHook( void )
{
    /* The malloc failed hook is enabled by setting
    configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

    Called if a call to pvPortMalloc() fails because there is insufficient
    free memory available in the FreeRTOS heap.  pvPortMalloc() is called
    internally by FreeRTOS API functions that create tasks, queues, software
    timers, and semaphores.  The size of the FreeRTOS heap is set by the
    configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h. */
    printf("Entered malloc failed hook");
    for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char
*pcTaskName )
{
    ( void ) pcTaskName;
    ( void ) pxTask;

    /* Run time stack overflow checking is performed if
    configconfigCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  This hook
    function is called if a stack overflow is detected.  pxCurrentTCB can be
    inspected in the debugger if the task name passed into this function is
    corrupt. */
    for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationIdleHook( void )
{
volatile size_t xFreeStackSpace;

    /* The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1 in
    FreeRTOSConfig.h.

    This function is called on each cycle of the idle task.  In this case it
```

```
   does nothing useful, other than report the amount of FreeRTOS heap that
   remains unallocated. */
   xFreeStackSpace = xPortGetFreeHeapSize();

   if( xFreeStackSpace > 100 )
   {
       /* By now, the kernel has allocated everything it is going to, so
       if there is a lot of heap remaining unallocated then
       the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
       reduced accordingly. */
   }
}
/*-----------------------------------------------------------*/

static void prvSetupHardware( void )
{
   /* Ensure all priority bits are assigned as preemption priority bits.
   http://www.freertos.org/RTOS-Cortex-M3-M4.html */
   NVIC_SetPriorityGrouping( 0 );

   // TODO: Setup the clocks, etc. here

   // Enable GPIOC Clock
   RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
       // Enable ADC1 Clock

   GPIO_InitTypeDef GPIO_config;
   //output pins
   GPIO_config.GPIO_Pin = red_light | green_light | yellow_light ;
   GPIO_config.GPIO_Mode = GPIO_Mode_OUT;          // Output
   GPIO_config.GPIO_OType = GPIO_OType_PP;         // Push Pull
   GPIO_config.GPIO_PuPd = GPIO_PuPd_DOWN;         // Pull Down
   GPIO_config.GPIO_Speed = GPIO_Speed_50MHz;      // Speed good enough for
shift reg
   GPIO_Init(GPIOC, &GPIO_config);

   //configure shift register output
   GPIO_InitTypeDef GPIO_config2;
   GPIO_config2.GPIO_Pin = shift_reg_clock | shift_reg_reset |
shift_reg_data;
   GPIO_config2.GPIO_Mode = GPIO_Mode_OUT;
   GPIO_config2.GPIO_OType = GPIO_OType_PP;
   GPIO_config2.GPIO_PuPd = GPIO_PuPd_NOPULL;
   GPIO_config2.GPIO_Speed = GPIO_Speed_50MHz;
   GPIO_Init(GPIOC, &GPIO_config2);
   ADC_setup();
}

//technically not polling, I am not changing the name
static uint16_t poll_adc_function(void){
```

```
   uint16_t converted_data;
   // 1. Trigger the conversion
   ADC_SoftwareStartConv(ADC1);
   // 2. Wait for the End-of-Conversion flag
   while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
   // 3. Read the result (0-4095)
   converted_data = ADC_GetConversionValue(ADC1);
   return converted_data;
}


//**********
static void ADC_setup(void){
   // ADC1 is on APB2
   RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
   // The GPIO pin (PC3) still needs AHB1
   RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
   GPIO_InitTypeDef GPIO_InitStruct;
   GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;      // Analog mode
   GPIO_InitStruct.GPIO_Pin  = GPIO_Pin_3;         // PC3
   GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;   // No pull resistor
   GPIO_Init(GPIOC, &GPIO_InitStruct);
   ADC_InitTypeDef ADC_InitStruct;

   //  ADC_StructInit(&ADC_InitStruct);
   ADC_InitStruct.ADC_ContinuousConvMode = DISABLE;
   ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
   ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b;
   ADC_InitStruct.ADC_NbrOfConversion = 1;
   ADC_InitStruct.ADC_ScanConvMode = DISABLE;
   ADC_InitStruct.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T1_CC1;
   ADC_InitStruct.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
   ADC_Init(ADC1, &ADC_InitStruct);
   // Configure Channel 13 (PC3), rank 1, 84-cycle sample time
   ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1,
                        ADC_SampleTime_84Cycles);
   //  ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1,
   ADC_Cmd(ADC1, ENABLE);  // Power on ADC1
   //  ADC_SoftwareStartConv(ADC1);
}
```